

AI 技术应用验证报告

一、验证场景与目的

1.1 验证场景

本次验证聚焦于一个具有代表性的中文软件需求工程场景，即使用大语言模型完成软件需求规格说明书（SRS）的自动生成。具体而言，验证场景设定如下：

在真实的软件项目启动初期，需求工程师通常需要面对一份或多份来源各异的需求素材，这些素材可能包括产品经理撰写的项目背景说明、用户访谈纪要、市场调研摘要以及既有系统的功能描述文档。需求工程师需要从这些非结构化或半结构化的文本中，识别并提炼出系统应当实现的功能需求与应当满足的非功能需求，并按照行业或企业内部的 SRS 模板，将其组织为一份结构完整、表述规范、可被后续设计与开发团队直接使用的软件需求规格说明书。

在传统工作方式下，这一过程高度依赖人工阅读、归纳、分类与撰写，工时投入大、格式一致性差，且对个人经验依赖较强。本验证将该场景作为最小可行验证对象，尝试在无人介入或仅做最小复核的前提下，由 LLM 与配套流水线自动完成从原始文本到结构化 SRS 的端到端生成。

所选定的具体验证素材是一份围绕"校园集市"校内信息平台的项目说明文档。通过在该场景下完成端到端运行，本验证旨在观察 LLM 在受限输入条件下是否具备独立产出可用 SRS 初稿的能力，并据此判断该方案在更复杂、更大规模的真实项目中是否具备推广价值。

1.2 验证目的

基于论文 REQINONE 提出的 SRS 自动生成流水线，以最小可行方式完成端到端验证，旨在回答以下三个层面的核心问题：

- AI 工具在中文需求工程场景下能否替代人工完成**需求提取与功能/非功能分类**？
- AI 辅助方式相比传统人工方式，**效率与质量**差异如何？
- 当前流水线在落地过程中存在哪些**可改进点**？

二、工具选型

2.1 基座模型

本验证选用 gpt-3.5-turbo 作为基座大语言模型。选择该模型主要基于以下几方面的综合考量。

第一，从能力匹配度来看，gpt-3.5-turbo 在通用自然语言理解与生成任务上已经具备较为稳定的表现，能够较好地完成需求摘要、关键句抽取、需求条目改写、功能与非功能分类等本验证所涉及

的核心子任务。

第二，从成本与可获得性来看，gpt-3.5-turbo 的单次调用价格显著低于更高级别的模型。在本次端到端验证中，流水线需顺序触发摘要、抽取、逐条分类等多轮 LLM 调用，单次运行的总体 token 消耗对成本较为敏感。选用 gpt-3.5-turbo 既能在合理预算内完成多次重复实验与对比，又便于在后续工程化阶段评估其规模化调用下的整体成本结构。

第三，从验证定位来看，本次验证的核心目的是评估"流水线 + 提示词"方案本身的可行性，而非横向对比不同基座模型的能力上限。因此在最小可行验证阶段，采用一个具备代表性与可用性的中等能力模型更具合理性。若后续需要进一步提升抽取质量或分类准确率，可在不改变流水线结构的前提下，平滑替换为更强模型，从而形成可对比、可升级的工程路径。

综合以上因素，gpt-3.5-turbo 是本次验证阶段在能力、成本、成熟度与扩展性之间取得平衡的合理选择。

2.2 pipeline框架

本验证的流水线结构以论文 REQINONE 所提出的 SRS 自动生成框架为蓝本。REQINONE 是近年提出的、以大语言模型为核心驱动的需求工程自动化方案，其核心思想是将 SRS 的生成过程拆解为若干相互衔接、可独立验证的子任务节点，并通过状态对象在节点间传递中间结果，从而实现从原始需求文本到结构化 SRS 文档的端到端自动化。该框架为本验证提供了清晰的节点划分参考与状态流转模型，是本次实现的基础。

三、执行过程

3.1 输入数据

项目相关的基本信息，主题为"校园集市"校内信息平台：

本项目为"校园集市"系统开发，旨在为校内学生提供一个集信息发布、资源共享、互动交流于一体的线上平台。平台主要包括帖子管理、评论互动、点赞支持、关键词搜索、个人信息管理等功能.....

3.2 流水线节点执行记录

顺序	关键提示词	主要行为
1	init	初始化 SRSState 字段
2	summary	顺序生成引言 / 利益相关者 / 用例 / 术语表 共 4 个章节
3	extract	一次性从原文提取结构化需求，输出 [REQ_n]...[/REQ_n] 块

4	parse	3 级回退正则：标记块 → 句式扫描 → 行级回退
5	classify	对每条需求独立调用 LLM，判定 Functional 或 11 类 NFR 之一
6	validate	统计提取数 / 分类数 / 章节完整性，产出 validation_report
7	assemble	拼装并落盘

3.3 实际运行结果

```
{
  "validation_report": {
    "extracted_count": 12,
    "functional_count": 7,
    "non_functional_count": 5,
    "has_introduction": true,
    "has_stakeholders": true,
    "has_use_cases": true,
    "has_glossary": true,
    "passed": true,
    "warnings": []
  }
}
```

extracted_requirements 摘录（12 条，节选前 5 条）：

- 1. 平台应当具有帖子管理功能，当用户发布、编辑、删除帖子时
- 2. 系统应当支持评论互动功能，当用户对帖子进行评论时
- 3. 平台应当具有点赞支持功能，当用户对帖子进行点赞时
- 4. 系统应当具有关键词搜索功能，当用户需要搜索特定内容时
- 5. 平台应当具有个人信息管理功能，当用户需要编辑、查看个人信息时

四、结果对比：传统人工 vs AI 辅助

4.1 传统人工方式

以 1 名中级需求工程师为基准，编写一份同等规模的中文 SRS：

步骤	估算工时
----	------

需求访谈 / 资料收集	1 工作日
需求条目撰写	1.5 工作日
功能 / 非功能 11 分类	0.5 工作日
利益相关者 / 用例 / 术语表	0.5 工作日
评审与修改	0.5-1 工作日
合计	3.5-4.5 工作日

4.2 AI 辅助方式

步骤	估算耗时
准备输入文本	5 分钟
summary 4 次调用	约 60-90 秒（4 × 单次 LLM 调用）
extract 1 次调用	约 15-25 秒
classify 10 次调用	约 50-80 秒（每条独立调用）
validate + assemble	< 1 秒（本地计算）
人工复核 + 微调	10-15 分钟
合计	15-20 分钟

4.3 量化对比表

维度	传统人工	AI 辅助	变化
端到端耗时	3.5-4.5 工作日	15-20 分钟	↓ 约 95%
章节完整度	易遗漏术语表	4/4 章节 100% 覆盖	↑
格式一致性	依赖个人习惯	由 assemble 强制统一	↑

单次 LLM 调用成本	—	约 15 次调用，估 $\approx$ 0.1–0.3 元（gpt-3.5-turbo + 国内代理）	—
可复用性	低（个人知识）	高（流水线即代码）	↑

五、改进建议

5.1 提示词工程（短期，最易落地）

针对 `CLASSIFICATION_SYS` 增加**对比样例与判定顺序**：

判定顺序（严格按此顺序匹配）：

1) 若涉及"安全/加密/权限/验证码/登录鉴权" → Security

2) 若涉及"性能/响应时间/并发/吞吐/检索速度" → Performance

3) 若涉及"易用/学习成本/操作步骤数/用户友好" → Usability

4) 若涉及"界面/视觉/风格/外观" → Look & Feel

5) 若涉及"可用率/SLA/容灾/故障恢复" → Availability / Fault Tolerance

6) 其它明确给出系统应"做什么"的动作 → Functional

请先输出一行 "label: <类型>", 再输出一行 "evidence: <原句中的关键词>"。

5.2 模型与温度（中期）

- 对分类任务保留 `temperature=0`；对摘要/用例生成任务允许 `temperature=0.3` 提升多样性。
- 启用 `bind(logprobs=True)`，对低置信度判定结果回退到"未分类"桶，由人工补录。

5.3 工程化（中期）

- 在 `classify` 之后、`validate` 之前增加 `node_review` 节点，对低置信度条目自动标注 `needs_human_review=true`。
- 在 `assemble` 之前增加**覆盖率检查**：若 `non_functional_count / extracted_count < 0.1`，触发重跑而非直接落盘。

5.4 流程治理（长期）

- 人机分工**：AI 只生成"初稿 + 候选分类 + 证据句"，最终 SRS 由人工确认。
- 可追溯性**：每次运行产物（`Generated_SRS_CN.md` + `pipeline_state_cn.json` + LLM 原始输出）必须进入版本管理，便于审计与回滚。

六、小结

本次验证围绕"以LLM为核心驱动的软件需求规格说明书自动生成"这一主题，以 REQINONE 流水线为蓝本、以 gpt-3.5-turbo 为基座模型、以"校园集市"项目说明文档为输入素材，完成了从原始文本到结构化 SRS 文档的端到端最小可行验证。

综合来看，本次验证支持如下结论：第一，AI 辅助 SRS 生成已经从"概念可行"进入"工程可用"阶段；第二，最小可行流水线已经具备在真实项目中作为需求工程师高效辅助工具的条件，前提是配套相应的提示词工程、置信度回退与人工复核机制；第三，从"可运行"走向"可生产"，需要在工程化、流程治理与持续评估三个方向上持续投入，单点优化无法解决系统性问题。